

1) Intro

We will focus on **kathara commands**

Each device of the emulated network is a docker container. Each container has a terminal window, a memory, a file system and most importantly some network interfaces

Kathara v Commands

These allow for the configuration of single devices:

```
kathara vstart -n name --eth N:X M:Y
```

This starts the device with specified name with the specified interfaces. Here is a quick rundown:

- **kathara vstart:** starts device
- **-n name:** gives specified name to device
- **--eth N:X M:Y** specifies 1 (in this case 2) or more interfaces connected to the said domain
 - N:X is ethN on domain X
 - usually we start naming them from 0 and the domain from A

```
kathara vclean -n name
```

Stops device with specified name

Kathara l Commands

These are used to set up more complex labs:

- **lstart:** starts a lab
- **lclean:** halts all devices of the lab
- **lrestart:** halts and restarts entire all devices
- **linfo:** provides info about lab

The lab settings are stored into a lab.conf file that stores a list of type

```
name[interface]=domain  
  
name1[interface1]=domain1  
name1[interface2]=domain2  
  
...
```

Where it stores for each machine (name) what interface (interface) is connected to what domain (domain). Notice how the second device (name1) has two interfaces on two different domains.

Moreover each device can have a name.startup file that stores a code that is run at the start of each device: For example:

```
ifconfig eth0 10.0.0.1/24 up  
/etc/init.d/zebra start
```

Remark (ifconfig).

The normal syntax for ifconfig is

```
ifconfig ethN ethIP netmask netmaskIP broadcast broadcastIP up
```

Katharà Global Commands

These are for management:

- check: checks system environment
- connect: connect to a running Katharà machine
- list: shows all running containers
- settings: edits Katharà settings
- wipe: delete all Katharà machines and links

2) Routing

In these exercises you will get a network scheme and your aim is to make all the devices communicate (ping) with each-other (usually a static routing without default gateways). Recall that in bigger network a **direction of packet flow must be specified**, otherwise some packets will bounce between two hosts and never reach the destination.

Here is a rundown of some useful commands:

Route

```
route
```

This will list the entire routing table of the device

```
route add -net netAddr/netmask gw nextHopIP dev ethN
route add -net netAddr netmask netmaskIP gw nextHopIP dev ethN
```

This adds the specified network+netmask to the routing table. It will get routed through the selected interface directed to the nextHopIP specified. Here is a quick rundown:

- **route add -net:** specifies a network is added to the routing table
- **netAddr/netmask OR netAddr netmask netmaskIP:** now you can specify the network using the Network address and
 - /netmask: with the mask in decimal (192.168.1.7/24)
 - netmask IP: with the netmask expressed in ip format (192.168.1.7 netmask 255.255.255.0)
- **gw:** this specifies the IP of the next hop (not the IP of the interface)
- **dev:** this specifies the name of the interface (usually of type ethN with N a number)

```
route add default gw nextHopIP dev ethN
```

This will add the default gateway (usually not allowed during exams). That is identical to writing

```
route add -net 0.0.0.0/0 gw nextHopIP dev ethN
```

```
route del -net netAddr netmask netmaskIP gw nextHopIP
```

This will delete from the routing table the entry with the specified network address, netmask and gateway

IP

```
ip a
```

This logs all interfaces and their IP. Notice it also lists 127.0.0.1 as the "internal" interface

```
ip route show
```

Logs routing table

Ping

```
ping dstIP
```

This will ping the destination and, if it is reachable, it will log it. Otherwise an error or nothing is returned

Traceroute

```
traceroute dstIP
```

This will log every hop taken to reach dstIP. Useful for debugging broken paths and finding missing links.

Tcpdump

```
tcpdump -i eth0
```

This logs in the terminal the actions taken by the tcp protocol

```
tcpdump -i eth0 -w /hosthome/capture.pcap
```

This doesn't log the output but writes (-w) it in the specified directory and with specified file name

```
tcpdump -t -n -i eth0
```

this sniffs on the interface with no protocol name, no DNS, and no timestamps all while showing mac addresses. This is a concatenation of -t (no timestamp), -e (show MAC) -n (no ip) -i (interface)

ARP

```
arp -a
```

Used to debug arp (usually not needed)

3) Firewall

Start apache2 and ssh

```
/etc/init.d/apache2 start  
/etc/init.d/ssh start
```

To give root privileges to ssh do:

```
nano /etc/ssh/sshd_config
```

and change the line

PermitRootLogin prohibit-password

to

PermitRootLogin yes

Then restart:

```
/etc/init.d/ssh restart
```

And set a password using

```
passwd
```

Access apache2 and ssh

For apache2 we have:

```
wget http://dstIP
```

This will test the connection

```
curl http://dstIP
```

This will log the page

```
links 127.0.0.1
```

This accesses the page on local machine, use q to exit

For ssh we have:

```
ssh root@dstIP
```

Where we then write "yes" and then the password selected before. To **exit** press **control-D**

iptables

By default everything is set to ACCEPT

```
iptables --table filter --flush
iptables --table filter --delete-chain
```

This will clear the table "filter".

- **table filter:** works on the default **filter** table

```
iptables -t filter -D FORWARD 2
```

This deletes a row (the second) in the filters table FORWARD append. Remember we start counting from 1

```
iptables -L
iptables -t filter -L -n -v
```

shows filters table

- -L: list rules
- -n: don't resolve names (faster, shows IPs/ports)
- -v: verbose (shows packet/byte counts, interfaces, etc.)
- -t filter: shows filter table

```
iptables --table filter --policy INPUT DROP
iptables --table filter --policy OUTPUT ACCEPT
iptables --table filter --append INPUT --in-interface lo --jump ACCEPT
```

Default policies for the input and output chains of the filter table. The third line avoids the breaking of local precesses.

- **in-interface lo:** matches packets **coming in on the lo interface**, which is the **loopback interface** (127.0.0.1 / localhost).

```
iptables --table filter --policy FORWARD DROP
```

This also disconnects lan from internet

```
iptables --table filter --append FORWARD --in-interface ethLAN --out-interface ethINTERNET --jump ACCEPT
```

This allows **new connections** from LAN to INTERNET

```
iptables --table filter --append FORWARD --in-interface ethINTERNET --out-interface ethLAN --match state --state ESTABLISHED,RELATED --jump ACCEPT
```

This allows **return traffic** from INTERNET to OUTSIDE if the connection was existing or related

```
iptables --table filter --append INPUT --protocol icmp --icmp-type echo-request --jump ACCEPT
```

Allows **ping requests** (icmp, type echo-request).

- **append INPUT:** appends a rule to the **INPUT chain** (packets coming *into* the device).
 - **protocol icmp:** targets **ICMP packets** (used for network diagnostics).
 - **icmp-type echo-request:** specifically matches **ping requests** (not replies or other ICMP types).
 - **jump ACCEPT:** if a packet matches this rule, **accept it**.
-

```
iptables --table filter --append INPUT --protocol tcp --destination-port 22 --jump ACCEPT
```

Allows **ssh requests** (TCP, port 22).

- **protocol tcp:** accept tcp connections
- **destination-port 22:** on port 22

```
iptables --table filter --append INPUT --protocol tcp --destination-port 22 --in-interface ethLAN --source srcIP --jump ACCEPT
```

This allows for ssh to be only accessed by LAN. Moreover it has to come from the specific srcIP.

```
iptables --table filter --append INPUT --match state --state ESTABLISHED,RELATED --jump ACCEPT
```

Usually used for IP packets and return traffic

- **match state:** uses the **state module**, which tracks connection state.
- **state ESTABLISHED,RELATED:** applies this rule to:
 - **ESTABLISHED:** packets that are part of an already established connection
 - **RELATED:** packets related to an existing connection (e.g. FTP data after FTP control)

```
iptables --table filter --append FORWARD --protocol tcp --destination-port 80 --in-interface ethINTERNET --out-interface ethLAN --destination dstIP --jump ACCEPT
```

This allows for **apache (http) requests** (tcp, port 80) to go into LAN from INTERNET to the specific dstIP

3.2) Recap:

What do the flags for iptables do?

```
--table filter  
-t filters
```

selects what table (filters) to act on

```
--policy TABLE ACCEPT/DROP  
-P TABLE ACCEPT/DROP
```

```
--append INPUT/FORWARD/OUTPUT  
-A INPUT/FORWARD/OUTPUT
```

specifies what chain to act on

```
--in-interface ethIN --out-interface ethOUT  
-i ethIN -o ethOUT
```

specifies in and out interfaces for the rule to act on.
Recall the **special interface "lo"** that is the local interface

```
--match state --state ESTABLISHED,RELATED  
-m state --state ESTABLISHED,RELATED
```

The values for --state can be NEX, ESTABLISHED, RELATED, INVALID

```
--protocol tcp --destination-port 22  
-p tcp --dport 22
```

match only for tcp packets on specific dst port. recall that these services (usually) work on these ports

- ssh: 22
- http (apache): 80

```
--protocol icmp --icmp-type echo-request  
-p icmp --icmp-type echo-request
```

Allows pings

```
--jump ACCEPT  
-j ACCEPT
```

Once this rule is encountered directly ACCEPT/DROP

NAT

In this case, connectivity is limited, as the LANs are on different address spaces and they cannot reach each other. We now set up R1 to perform NAT on outgoing packets:

```
iptables --table nat --append POSTROUTING --source 192.168.10.0/24 --out-interface  
eth1 --jump SNAT --to 161.175.30.253
```

Packets from inside the LAN can now reach the outside. Replies will be sent to R1's public address, but the rule will detect "conversations" and forward the appropriate packets.

Now we need to expose LANWS to the Internet, as it needs to provide a public service:

```
iptables --table nat --append PREROUTING --protocol tcp --destination 161.175.30.253  
--destination-port 80 --in-interface eth1 --jump DNAT --to 192.168.10.240:80
```

This command forwards any web request sent to R1 to LANWS, on the same port: in this way, WS can now reach LANWS's web service, while it still cannot ping any machine inside the LAN.

3.3) Theory

A firewall can be

- **Physical:** a separate device on network edge
- **Logical:** a process on the machine

Moreover, based on what it is doing it is:

- **Stateless:** Inspects packets individually, not as part of an ongoing connection; typically checks the protocol type and the source and destination addresses and ports, dropping (i.e., silently discarding) or rejecting (i.e., discarding with an error message) packets that do not match the whitelist criteria.
- **Stateful:** Implements more complicated rules: the inspection capability now reaches the transport layer, including the possibility to:
 - keep track of previously seen packets

- consider connections and whether packets are part of an ongoing “conversation” or is the beginning of a new one
- speed up process by making connection-level decisions with no full packet inspection.

There are two main goals:

- Ingress filtering: safeguard inner network by inspecting incoming traffic
- Egress filtering: prevent users to reach certain services or send certain data by inspecting outgoing traffic

The Linux tool **iptables** is a powerful packet filter, and it is the one we are going to use. It acts on **NetFilter**

NetFilter

It is structured on two levels: there are four tables which contain chains. The tables are called **filter**, **nat**, **mangle** and **raw**

FILTER TABLE

Used for basic packet filtering: has the following 3 chains:

- INPUT
- OUTPUT
- FORWARD

Every chain has a list of rules defining some matching criteria and target. Every packet is treated according to the first matching rule it encounters or the default policy if it matches no rules. The most common targets are:

- ACCEPT: accepts the packet
- DROP: silently discards it
- REJECT: discards it and sends an error message to the source
- user-defined-chain: redirects it to a specified user-defined chain
- RETURN: returns from the user-defined chain

NAT TABLE

This is used for NAT operations and has the 3 following chains:

- PREROUTING
- OUTPUT
- POSTROUTING

In this table, targets are packet manipulation operations, which often change the source and destination address and port. The most common targets are:

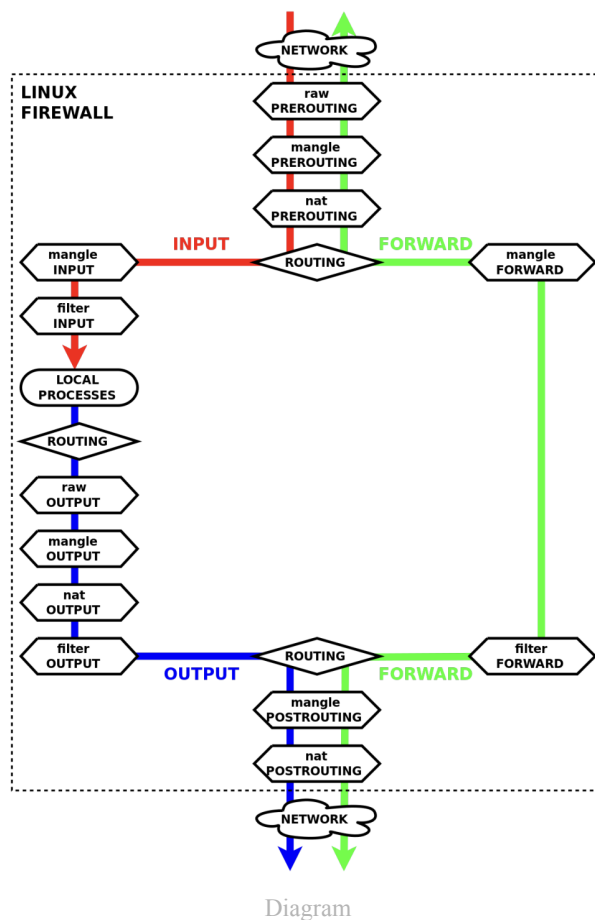
 - DNAT: changes the destination address and port of a packet
 - SNAT :changes the source address and peply, or Forwardort of a packet
 - MASQUERADE: works like SNAT, but it supports firewalls with dyanmic public IPs
 - REDIRECT: redirects the packet to the firewall

MANGLE TAB

Used for packet alterations (change TTL) but we won't use it.

RAW TABLE

Used to set up exceptions to the “conversation” tracking system but we won't use it



Diagram

Forward Stream Packets

Output stream packets are sent by the host that iptables is running on.

- The packet first enters one of the firewall network interfaces
- Then enters the PREROUTING chain of the nat table typically used for destination network address and port translation (DNAT) operations
- Now it's time to route the packet. (DNAT operations happens before routing decision and filtering so the packet can be routed and successively filtered accordingly to its "true" destination address. Since we have supposed the packet to belong to the forward stream it will be forwarded)
- Then the packet enters the FORWARD chain of the filter table where all filtering happens.
- If allowed the packet reach the POSTROUTING chain of the nat table typically used for source network address and port translation (SNAT) operations. (Note that filtering happens before SNAT operations take place so the packet contains its "true" source address (after SNAT operations all packets have the same source address so it is useless filter them based on that parameter)).
- At the end the packet exits on one of the network interfaces

Examples

Stateless:

Rule	Direction	Src. Addr.	Dest. Addr.	Protocol	Src. Port	Dest. Port	ACK	Action
A	Inbound	External	Internal	TCP		25		Permit
B	Outbound	Internal	External	TCP		>1023		Permit
C	Outbound	Internal	External	TCP		25		Permit
D	Inbound	External	Internal	TCP		>1023		Permit
E	Either	Any	Any	Any		Any		Deny

Example

This passes SMTP mail (port 25) but the sender might send from any port . If someone uses spoofing to enter the firewall by pretending it is from an internal network it is blocked. The service X11 is on port 6000 and thus it's traffic is allowed in and out.

Rule	Direction	Src. Addr.	Dest. Addr.	Protocol	Src. Port	Dest. Port	ACK	Action
A	Inbound	External	Internal	TCP	>1023	25		Permit
B	Outbound	Internal	External	TCP	25	>1023		Permit
C	Outbound	Internal	External	TCP	>1023	25		Permit
D	Inbound	External	Internal	TCP	25	>1023		Permit
E	Either	Any	Any	Any	Any	Any		Deny

Updated Example

This is now fixed since rule A blocks traffic incoming traffic that doesn't go to port 25. By also specifying the ack of B and D to be "YES" it blocks new TCP connections

Stateful:

Rule	Direction	Src. Addr.	Dest. Addr.	Protocol	Src. Port	Dest. Port	ACK	Action
A1	Inbound	External	Webserver	TCP	>1023	80	Any	Permit
A2	Outbound	Webserv.	External	TCP	80	>1023	Yes	Permit
B1	Outbound	Internal	External	TCP	>1023	80	Any	Permit
B2	Inbound	External	Internal	TCP	80	>1023	Yes	Permit
C1	Outbound	Internal	External	UDP	>1023	53	-	Permit
C2	Inbound	External	Internal	UDP	53	>1023	-	Permit
D	Either	Any	Any	Any	Any	Any	Any	Deny

Example

This allows HTTP (TCP 80) traffic by external hosts and allows internal hosts to initiate HTTP or DNS (UDP 53) while denying all other communications.

Rule	Direction	Source Address	Destination Address	Protocol	Source Port	Dest. Port	State	Action
A	Inbound	External	Webserver	TCP	>1023	80	New	Permit
B	Outbound	Internal	External	TCP	>1023	80	New	Permit
C	Outbound	Internal	External	UDP	>1023	53	New	Permit
D	Either	Any	Any	Any	Any	Any	Established	Permit
E	Either	Any	Any	Any	Any	Any	Any	Deny

Stateful Rules

For HTTP and DNS it allows new connection. Already established connections are also permitted

Proxy Firewall inspects traffic on the application layer and acts as an intermediary by impersonating the recipient. It is slow